Phase 03: Model Deployment & Security Automation

Advanced Data Analysis

Ayaan Hirsi

Table of Contents

Introduction 1
System Architecture Overview 2
High Level Architecture Diagram 2
Core Components 2
Client Input Layer 3
FastAPI IDS Service 3
Inference Feature Engineering 3
Binary Classification Model (Benign vs Malicious) 3
Attack-Type Model and Label Encoder 3
Threat Intelligence: AbuseIPDB + ipinfo 3
Decision & Enrichment Stage 4
JSON Response to Client 4
Discord Alerting (for Malicious Flows) 4
Deployment Setup 4
Environment 5
Running the Service 5
API Endpoints 5
POST /analyze - Single Flow Analysis 5
POST /analyze/batch — Batch Processing 5
Detailed Inference Pipeline 5
Input Validation 6
Feature Engineering 6
Binary Classification 6
Attack-Type Classification 6
Threat Intelligence Integration 6
Google Sheets Logging 6
Discord Alerting 7
Evaluation & Results 9
Normal Flow Example 9

Malicious Flow Evaluation....9
Error Handling & Edges Cases....10
Input Validation and Schema Enforcement....10
Handling External API Failures....10
Model Prediction Safety....11
Invalid or Unrecognized Protocol Values....11
Batch Processing Edge Cases....11
System Stability....11
Security Considerations....11
Credential Security....11
Secure API Access....12
Input Sanitization....12
Rate Limiting and Abuse Prevention....12
Model Safety....12
Logging and Privacy....12
Limitations & Future Work....12
Conclusion....13
References....14

Introduction

The goal of Phase 3 is to transition our machine learning intrusion detection models from Phases 1 and 2 into a full operational security microservice. Our earlier phases focused on exploratory analysis, feature engineering, baseline models, and transformers. This final phase emphasizes deployment, real-time inference, automation, threat intelligence, alerting, and reliability.

We built a FastAPI-based IDS service that accepts network flow data, performs ML predictions, enriches results with external threat intelligence sources, logs normal behavior for monitoring, and triggering alerts for malicious activity. The system mimics real-world SOC workflows, where detection engines feed insights into logging pipelines, dashboards, and analyst notification channels.

This report outlines the architecture, implementation, evaluation, automation workflows, security considerations, and limitations of the deployed system.

System Architecture Overview

The Intrusion Detection System developed in Phase 3 follows a modular, production-inspired architecture designed to support real-time interference, threat enrichment, and automated alerting.

High Level Architecture Diagram

The architecture is organized as a linear detection pipeline with conditional branching for malicious flows. It begins with incoming client requests and ends with a structured JSON response, and when required, an automated alert.

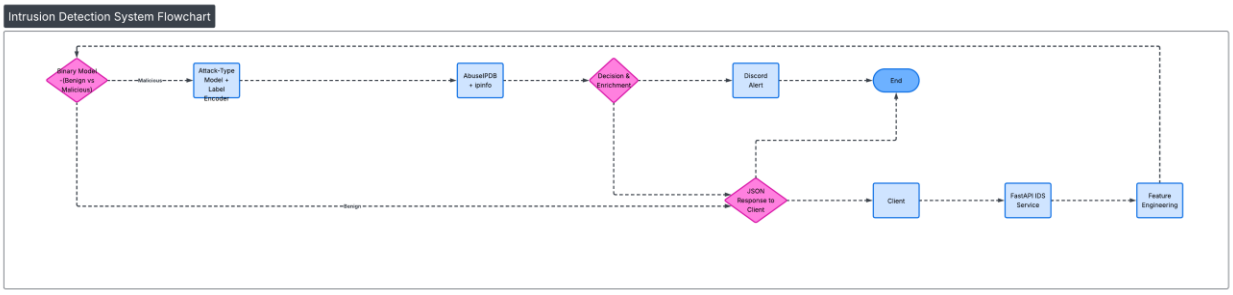


Figure 1: Intrusion Detection System Flowchart

Core Components

Client Input Layer

The system is accessed through HTTP POST requests issued by a client application such as Postman. The client sends a JSON object representing a single network flow to the FastAPI endpoint /analyze.

FastAPI IDS Service

The FastAPI application acts as the central controller for the IDS. Its responsibilities are the following:

- Validating the input using the FlowData schema
- Passing the flow to the feature engineering module
- Managing the prediction process for both binary and attack-type models
- Orchestrating enrichment steps such as reputation and geolocation lookups
- Sending alerts when malicious traffic is detected
- Returning a final structured JSON response to the client

This service ensures consistent routing of requests and integrates all downstream components.

Inference Feature Engineering

Before any prediction is made, the system preprocesses the incoming flow using a dedicated feature engineering module. This stage reconstructs all of the necessary derived features that were used during the model training, such as protocol encoding, byte and packet ratios, connection-state transformations, and numerical standardization. This step ensures that the deployed models receive inputs in the exact format they expect, preserving model reliability.

Binary Classification Model (Benign vs Malicious)

The first machine learning model classifies each network flow as either benign or malicious. This binary decision acts as the primary branching point in the architecture: Benign flows bypass the attack-type prediction branch, and Malicious flows proceed to further analysis and enrichment. This model is lightweight, fast, and optimized for real-time execution.

Attack-Type Model and Label Encoder

For flows predicted as malicious; a secondary classifier determines the specific attack category (such as: DDoS, MITM, scanning activity). The numeric output of this model is translated into a human-readable label using a corresponding label encoder. This enables the system to provide a detailed threat context rather than a simple binary verdict.

Threat Intelligence: AbuseIPDB + ipinfo

To enrich the prediction and support investigative value, the system integrates two external intelligence sources:

- AbuseIPDB: Provides a reputation score indicating whether the source IP has a history of malicious activity
- ipinfo: Returns geolocation and ASN/ISP details, enabling attribution and situational awareness

These enrichment steps operate independently of the ML models and can override or reinforce their predictions, creating a hybrid decision engine.

Decision & Enrichment Stage

This component fuses binary model output, attack-type classification, IP reputation, and geolocation metadata. The system consolidates this information into a final structured verdict that includes both quantitative and contextual insights. This combined result drives both the client response and alerting behavior.

JSON Response to Client

Regardless of the verdict, every flow results in a structured JSON response containing:

- verdict
- attack_type (if malicious)
- reputation score
- geolocation info

Discord Alerting (for Malicious Flows)

If a flow is determined to be malicious, the system sends a formatted alert to a dedicated Discord channel. The alert includes:

- Source and destination IPs
- Attack type
- Geolocation details
- Reputation score

This mirrors the behavior of SOC notification pipelines and provides immediate visibility into suspicious activity.

Deployment Setup

Environment

- Python 3.x
- Virtual environment (venv)
- All dependencies specified in requirements.txt:
- fastapi, uvicorn
- pandas, scikit-learn, joblib
- requests
- gspread, oauth2client
- discord-webhook

Running the Service

- source .venv/Scripts/activate
- python main.py
    - At startup, the models are loaded, Webhook URLs and API keys are initialized, and FastAPI listens on port 5000.

API Endpoints

POST /analyze - Single Flow Analysis

Features:

- Full feature engineering
- Model inference
- Reputation + geolocation enrichment
- Normal traffic logged to Sheets
- Malicious traffic triggers Discord alert

POST /analyze/batch — Batch Processing

Features:

- Supports large sets of flows
- Skips Google Sheets logging
- Sends summary alert to Discord
- Returns array of verdicts + attack types

Detailed Inference Pipeline

Input Validation

The Flowdata Pydantic model ensures fields match expected types such as IP addresses, ports, byte and packet counts, duration, and DNS/HTTP optional fields.

Feature Engineering

The apply_inference_engineering() function applies all the transformations needed to make the raw flow data compatible with the trained models. It maps protocol values to numeric codes (for example, tcp: 1, udp: 2, icmp: 3), computes ratio features such as byte_ratio and pkt_ratio, and performs one-hot encoding on connection states to create columns like conn_state_S0, conn_state_SF, and conn_state_REJ. Finally, all engineered features are cast to numeric types, and any missing or invalid values are replaced with 0 to avoid NaNs during inference.

Binary Classification

In the first prediction stage, the binary classifier takes the engineered feature vector and outputs a simple decision: 0 for normal traffic and 1 for malicious traffic. This initial verdict is used to decide whether a flow should stop at a benign classification or be passed on for deeper attack-type analysis and enrichment.

Attack-Type Classification

If and only if the binary classifier labels a flow as malicious, a second model is invoked to predict the specific attack category. This attack-type classifier outputs an integer label, which is then converted to a human-readable string (for example, DoS, DDoS, PortScan, or Recon) using a label encoder. This allows the system to provide more detailed information than just “malicious”.

Threat Intelligence Integration

To strengthen the decision-making process, the system integrates external threat intelligence. For each source IP, it queries AbuseIPDB to obtain an abuse confidence score; if the machine learning model predicts “normal” but the reputation score is greater than zero, the system can override the verdict and treat the flow as malicious, reducing potential false negatives. In addition, the service calls ipinfo to attach geolocation information such as country, region, city, and ISP/ASN, which enriches alerts and makes them more useful for security analysts.

Google Sheets Logging

For flows that are ultimately classified as normal, the system can append the original flow data as a new row in a Google Sheet using a service account. This optional logging step creates a simple record of benign activity that can later be used for baseline analysis, monitoring trends, or detecting changes in normal behavior over time.

Discord Alerting

When the final verdict for a flow is malicious—either directly from the model or due to a reputation-based override—the system sends a real-time alert to a dedicated Discord channel via a webhook. These single-flow alerts include key details such as the source and destination IPs, the predicted attack type, and the geolocation. In batch mode (when /analyze/batch is used), the system can also send a summary alert that reports how many flows were analyzed and how many were identified as threats, along with a short list of the most relevant malicious IPs.

Evaluation & Results

Normal Flow Example

To evaluate the behavior of the deployed IDS service on benign traffic, a normal network flow through the /analyze endpoint using Postman. The returned verdict was classified as “normal” and no attack type or geolocation was attached, which is the expected behavior for non-malicious flows. The model correctly identified the traffic as benign, and the enrichment modules showed there was no reputation risk associated with the source IP.

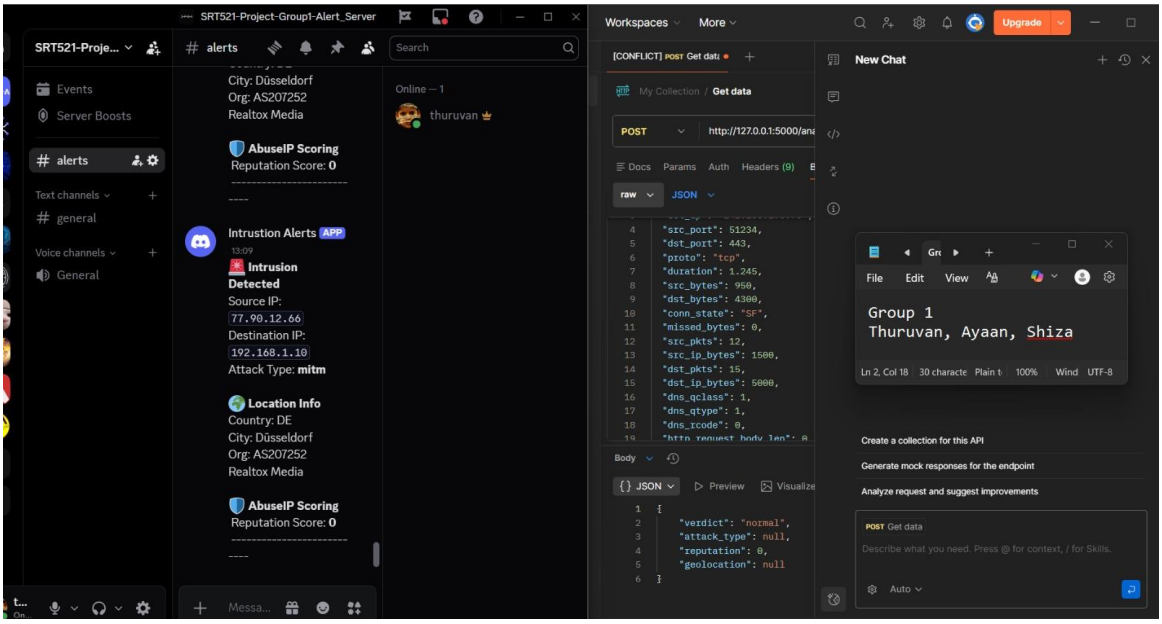


Figure 2: Normal Flow

Malicious Flow Evaluation

For testing purpose, a malicious flow example was tried. When sending a malicious sample, the API classified the flow as malicious, triggered the attack-type classifer, and produced a readable prediction such man in the middle. The malicious flow generated a real-time alert in Discord

containing all contextual details, source ip, destination ip, attack type, geolocation, and reputation. These alerts simulate the workflow of a real SOC team, providing immediate visibility into threats and supporting faster triage.

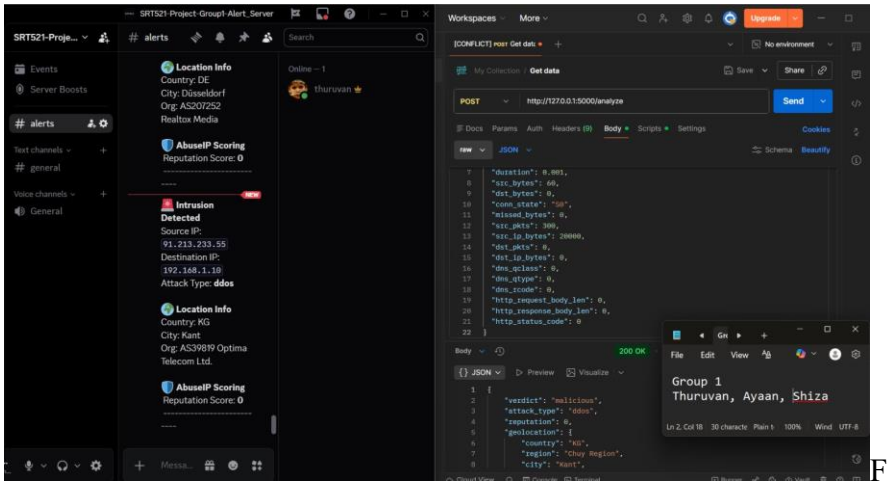


Figure 3: Malicious Flow

Error Handling & Edges Cases

A production-ready intrusion detection service must operate reliably even under unexpected or imperfect conditions. To ensure robustness, our Phase 3 system incorporates structured error handling and fallback mechanisms across every stage of the inference and enrichment pipeline.

Input Validation and Schema Enforcement

All incoming requests are validated using strict schema which is FlowData.

- Missing fields, incorrect types, or malformed JSON immediately generate a 422 Validation Error, preventing invalid data from reaching the model.
- This reduces the risk of unpredictable model behavior and protects against malformed or adversarial input.

Handling External API Failures

The system relies on two external services, AbuseIPDB and ipinfo for enrichment. To maintain reliability even when these services fail:

- **AbuseIPDB timeouts or quota errors:** the system inserts a fallback structure instead of raising expections, this ensures the main ML prediction flow remains uninterrupted.
- **Ipinfo Geolocation issues**: If the request fails, the service returns unknown values for country, region and city.

Model Prediction Safety

Machine learnign models can sometimes raise unexpected errors. The system handles these cases by:

- Wrapping prediction functions in try-except blocks
- Returning an explicit message to the user instead of crashing the server
- This prevents single malformed requests from taking down the entire API.

Invalid or Unrecognized Protocol Values

If a flow contains an unknown protocol:

- The protocol mapping function assigns a default numeric code
- The flow continues through the inference pipeline without interruption

This approach maintains compatibility with previously unseen protocols.

Batch Processing Edge Cases

While batch mode is optional, the system anticipates:

- Empty lists
- Mixed-validity inputs
- Out-of-range feature values

Any problematic rows are skipped with error messages, while the valid ones continue through.

System Stability

Most importantly, no error can crash the FastAPI application.
Every external dependency is decoupled, and model inference is isolated so that uptime is maximized.

Security Considerations

Given that this project simulates a real intrusion detection environment, security controls were integrated into the design of the deployment pipeline.

Credential Security

- API keys for AbuseIPDB and ipinfo should be stored in environment variables, not source files.
- This prevents accidental leakage through GitHub or shared devices.
- Service credentials should be rotated periodically as part of best practices.

Secure API Access

- The /analyze endpoint is designed for internal use and assumes a trusted network environment.
- In a production system, additional controls such as IP whitelisting, JWT authentication, or API gateways would be necessary to prevent misuse.

Input Sanitization

The Pydantic schema acts as a shield against:

- malicious JSON payloads
- type exploits
- high-volume garbage traffic that could degrade the model

This ensures the server processes only structured and meaningful data.

Rate Limiting and Abuse Prevention

AbuseIPDB has strict rate limits. To avoid violating them:

- The service only queries IP reputation when necessary
- Caching or backoff strategies could be added in future work

Model Safety

The ML models themselves operate on controlled numeric inputs, reducing attack surface for:

- adversarial example injection
- model poisoning
- inference-time manipulation

Logging and Privacy

Since no sensitive personal data is being stored or transmitted, the system aligns with basic privacy requirements. Still, logged inputs must not contain PII beyond network metadata.

Limitations & Future Work

Although the deployed IDS works effectively for real-time flow classification and alerting, several limitations remain:

Limitations

- The system does not integrate with enterprise SIEM tools.
- Detection is limited to a single API instance, no distributed or high-volume scaling.

- No real-time streaming pipeline such as Kafka or RabbitMQ.
- Feature engineering is static and may not generalize to evolving attack patterns.
- The system does not include explainability tools to help analysts understand predictions.

Future Work

- Add authentication (JWT or API keys) to secure the API.
- Store results in a database or SIEM backend for long-term analysis.
- Build a simple dashboard to visualize alerts and traffic patterns.
- Use more advanced or ensemble models to improve detection accuracy.
- Containerize the system using Docker/Kubernetes for scalable deployment.

Conclusion

Phase 3 successfully transforms our machine-learning IDS models into a functional real-world detection service. By combining FastAPI deployment, consistent feature engineering, threat-intelligence enrichment, and Discord alerting, the system can classify network flows and provide analysts with valuable context in real time.

This final phase demonstrates the full lifecycle of a security data science project from modeling and evaluation to deployment and operational use. The system provides a strong foundation that can be expanded into a more scalable, production-ready IDS in the future.

References

- Moustafa, N., Creech, G., & Slay, J. (2020). ToN-IoT: The First Cyber Dataset for IoT and IIoT Networks. UNSW Canberra Cyber. https://research.unsw.edu.au/projects/toniot-datasets
- AbuseIPDB. (2024). AbuseIPDB: IP Address Abuse Reports and Threat Intelligence.
- https://www.abuseipdb.com/
- ipinfo.io. (2024). IPinfo Geolocation API Documentation. https://ipinfo.io/
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/
- FastAPI. (2024). FastAPI: Modern, Fast (High-performance) Web Framework for Python.
- https://fastapi.tiangolo.com/
- Pydantic. (2024). Pydantic Documentation: Data Validation and Settings Management. https://docs.pydantic.dev/
- Uvicorn. (2024). Uvicorn ASGI Server Documentation. https://www.uvicorn.org/
- Discord Inc. (2024). Discord Webhooks Documentation. https://discord.com/developers/docs/resources/webhook
- Scarfone, K., & Mell, P. (2007). Guide to Intrusion Detection and Prevention Systems (IDPS). NIST Special Publication 800-94. https://csrc.nist.gov/publications/detail/sp/800-94/final